Carrera de Ciencia de Datos
e Inteligencia Artificial
FACULTAD DE INGENIERÍA

MÉTODOS EN CONJUNTO Y AJUSTE DE MODELOS

- **Bagging - pasting - out-of-bag**
- **Random Forests y Extra-Trees**
- **Métodos de potenciación (Boosting): AdaBoost, Gradient Boosting, XGBoost.**
- **Stacking y blending de modelos**

Ph.D. Lida Barba M., Docente Tutora.

¿Por qué usar mas de un método “simple” y no usar un solo modelo potente en machine learning?

- Proceso inspirado en la realidad social.
- Siempre es mejor el *trabajo en equipo*, la construcción conjunta genera una interdependencia positiva, donde el éxito del equipo depende del aporte de todos, motivando el compromiso individual y grupal.
- El *aprendizaje colaborativo* es potente porque transforma el proceso educativo de una actividad pasiva a una experiencia activa y social.
- Propicia el *constructivismo social*, porque los procesos se enriquecen desde la diversidad de perspectivas, es posible aprender enseñando, y por el desarrollo socioemocional, preparando a los individuos para los entornos laborales reales.

Ensemble Learning: Definition

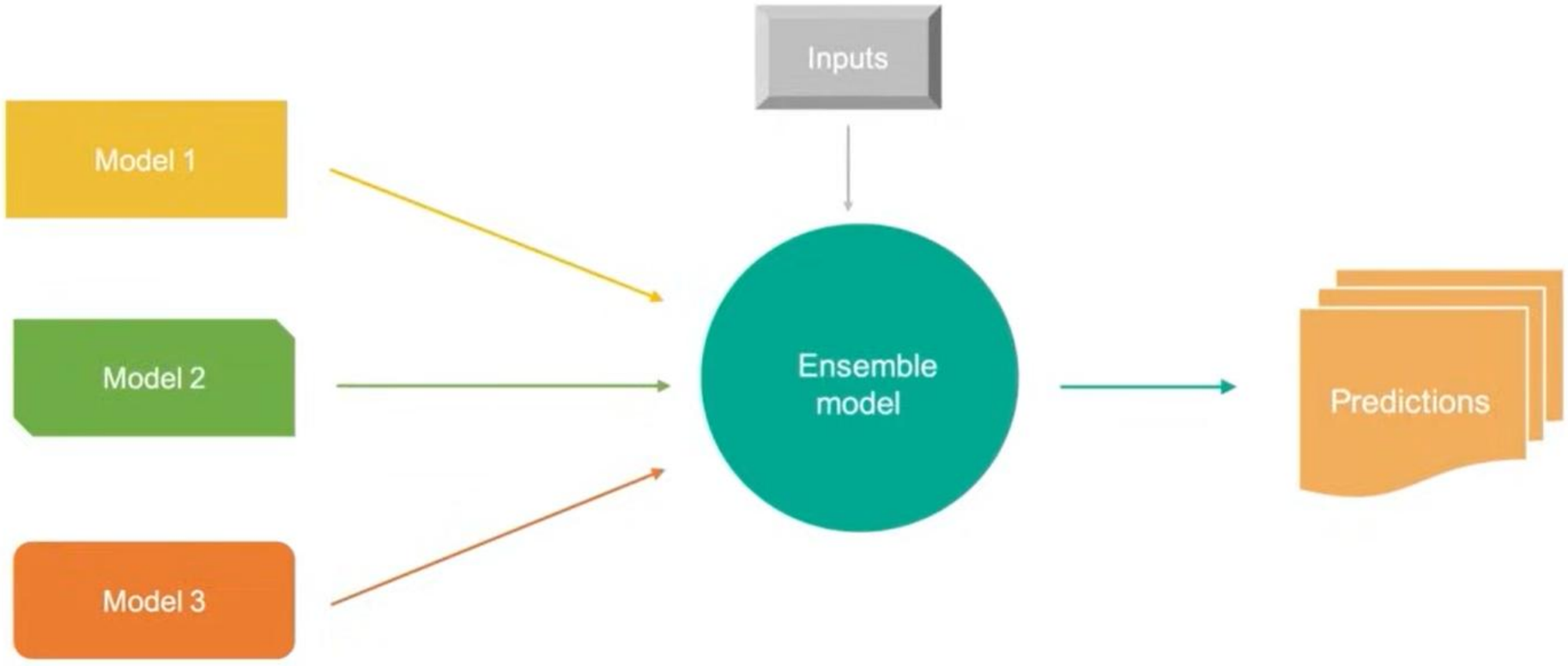
Certain models do well in modeling one aspect of the data, while others do well in modeling another

Learn several simple models and combine their output to produce the final decision

The combined strength of the models offsets individual model variances and biases

This provides a composite prediction where the final accuracy is better than the accuracy of individual models

Ensemble Learning: Working



Ensemble Learning: Significance

Ensemble model is the application of multiple models to obtain better performances than from a single model.

Robustness

Ensemble models incorporate the predictions from all the base learners

Accuracy

Ensemble models deliver accurate predictions and have improved performances

Ensemble Learning Methods

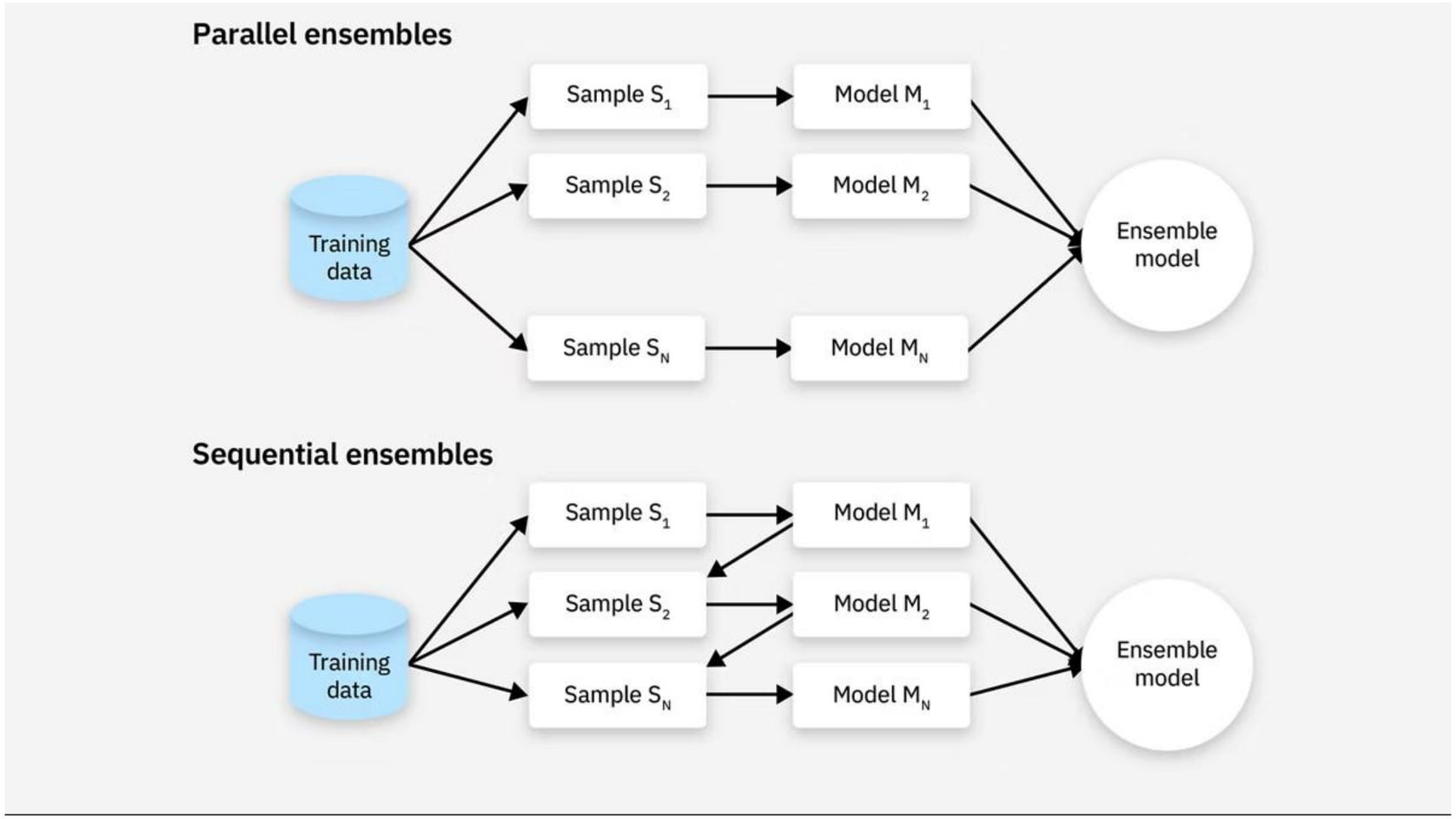
Combine all “weak” learners to form an ensemble

OR

Create an ensemble of well-chosen strong and diverse models

Ensemble models gain more accuracy and robustness by combining data from numerous modeling approaches.

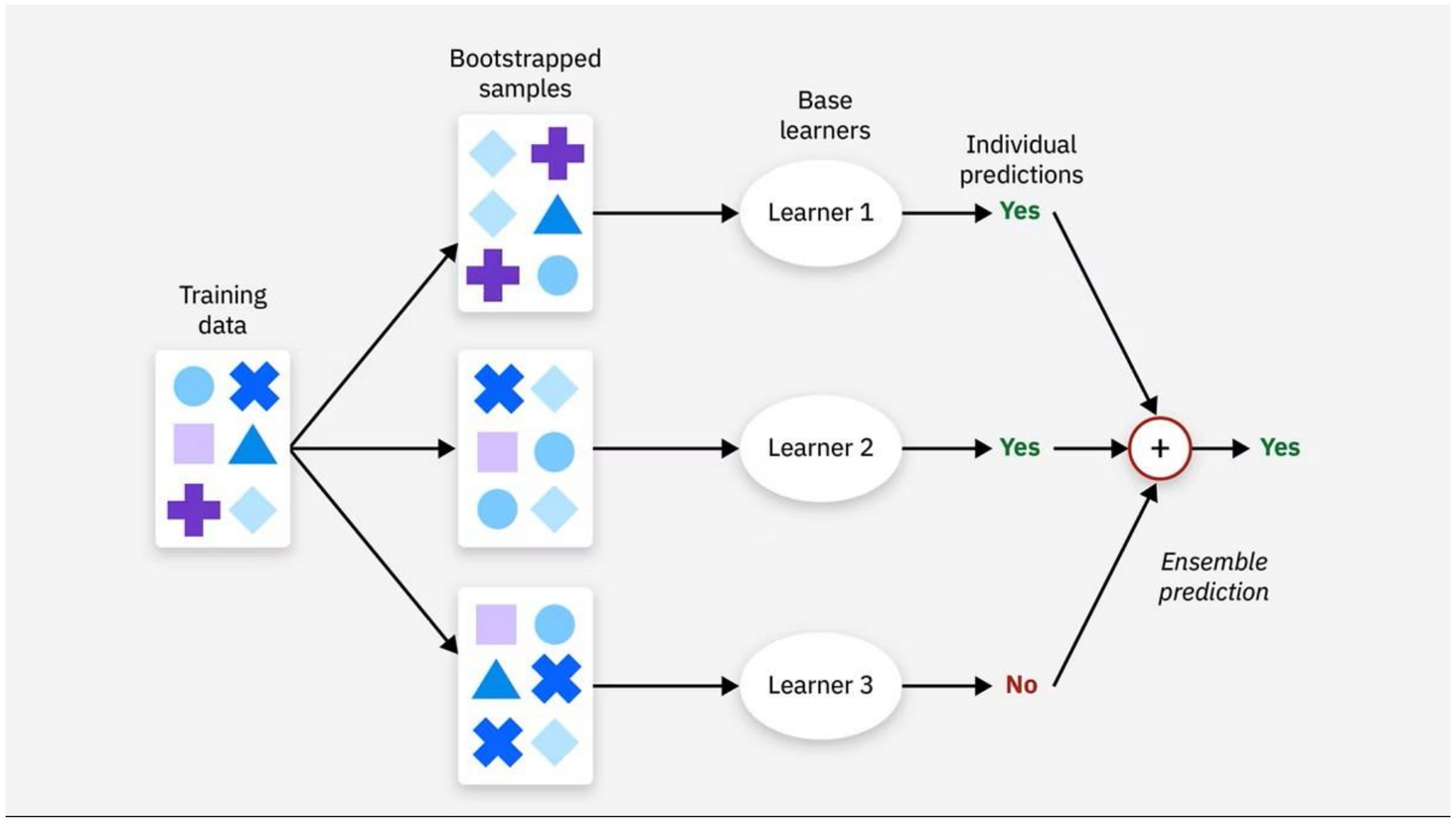
Tipos de aprendizaje en conjunto



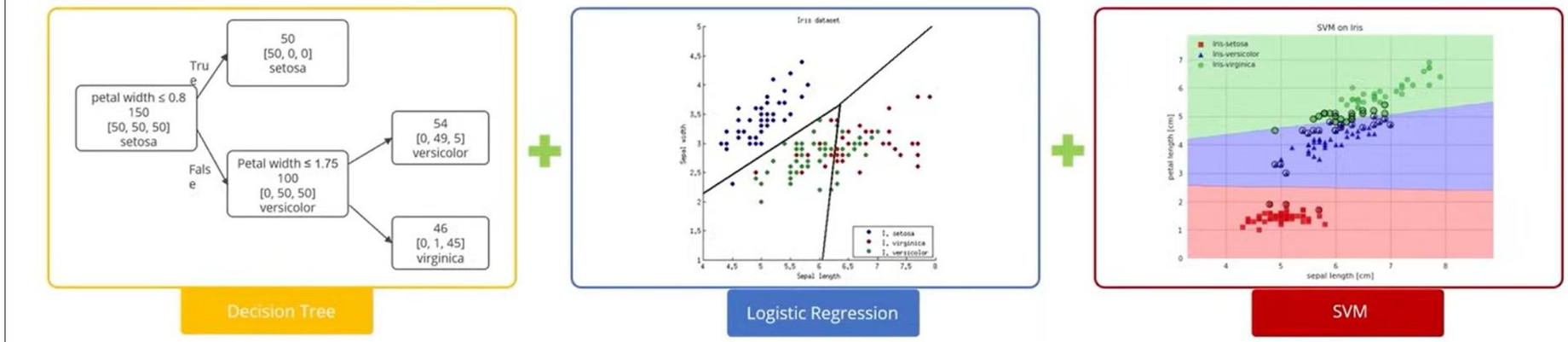
Los métodos paralelos se dividen a su vez en métodos **homogéneos** y **heterogéneos**. Los conjuntos paralelos homogéneos utilizan el mismo algoritmo de aprendizaje. Los conjuntos paralelos heterogéneos utilizan diferentes algoritmos para producir aprendices base.

Votación y agregación en modelos ensemble

Un método habitual para consolidar las predicciones en modelos ensemble consiste en utilizar la votación mayoritaria (moda) en problemas de clasificación, mientras que en problemas de regresión la predicción final se obtiene mediante el promedio o promedio ponderado de las salidas generadas por los modelos individuales.



Promedio simple y Promedio Ponderado



Averaging

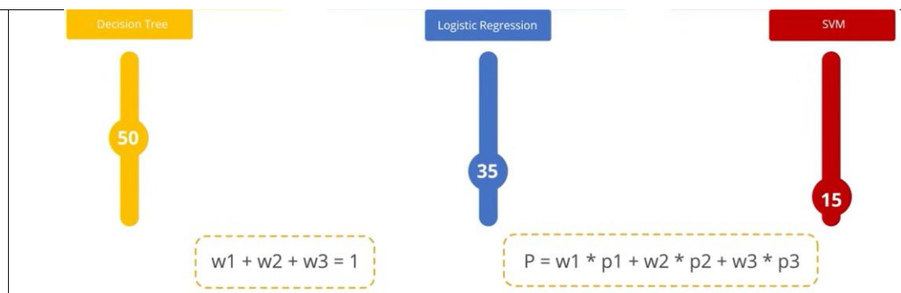
Ensemble prediction

$$P = \frac{p1 + p2 + p3}{3}$$

A class label is predicted using mode of member predictions.

The class probability is calculated as the argmax of the summed probabilities for each class label.

Equal weights are assigned to different models despite some models performing better than others.



Bagging

- **Bootstrapping** is a statistical resampling method that involves **randomly drawing samples with replacement** from a dataset to create many "new" samples, each the **same size as the original**.

- These samples are called bootstrap samples, and each contains some instances **multiple times** and may **omit others**.

- Multiple models (often of the **same type**) are trained on **different subsets** of the training data.

- These subsets are created by **randomly sampling** the original dataset with **replacement** (**bootstrapping**).

- Each model is **trained independently**, and their predictions are **combined** (e.g., by averaging for regression or majority voting for classification) to produce the final output.

Bagging

- Bagging helps **reduce variance** and prevent overfitting by leveraging the diversity among the models.
- Each **subset** is typically **smaller than** the original dataset, which means each individual predictor has **less data to learn from**.

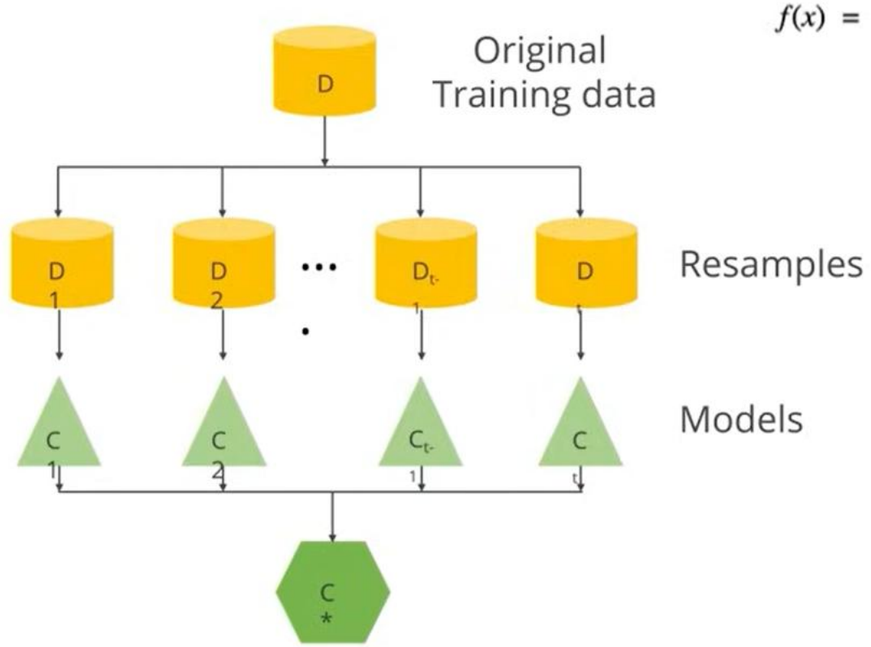
- Less data can lead to simpler models or models that **cannot fully capture** the underlying patterns in the data.
- This **increases the bias** of individual predictors compared to training them on the entire dataset.

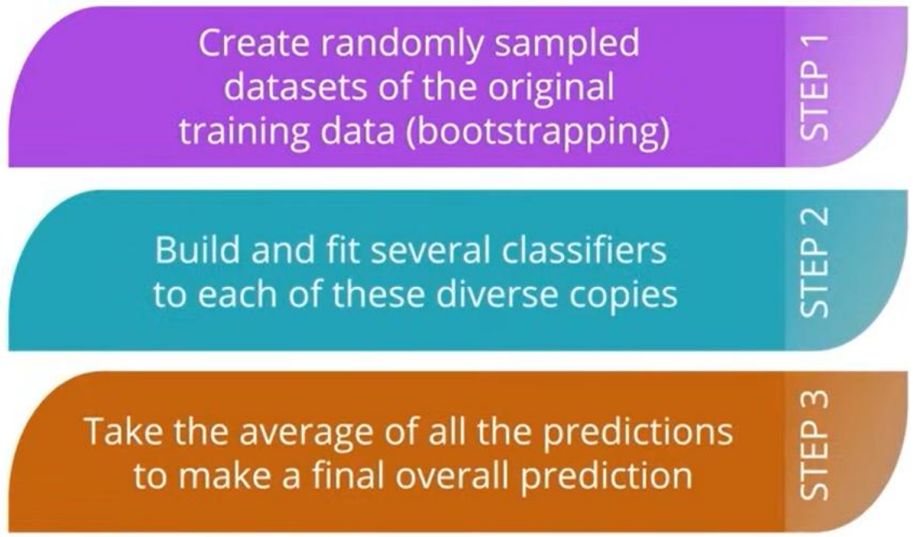
Variance Reduction

- Variance refers to how much a model's predictions would vary if trained on different data samples. In bagging, predictions from multiple predictors are averaged (or aggregated).
- This averaging reduces the impact of any one predictor's fluctuations (high variance) on the final prediction.
- Mathematically, averaging random variables reduces their variance, assuming the predictors' errors are not perfectly correlated.

Bagging or bootstrap aggregation **reduces variance** of an estimate by taking mean of multiple estimates.

$$f(x) = 1/M \sum_{m=1}^{M} f_m(x)$$





Pasting

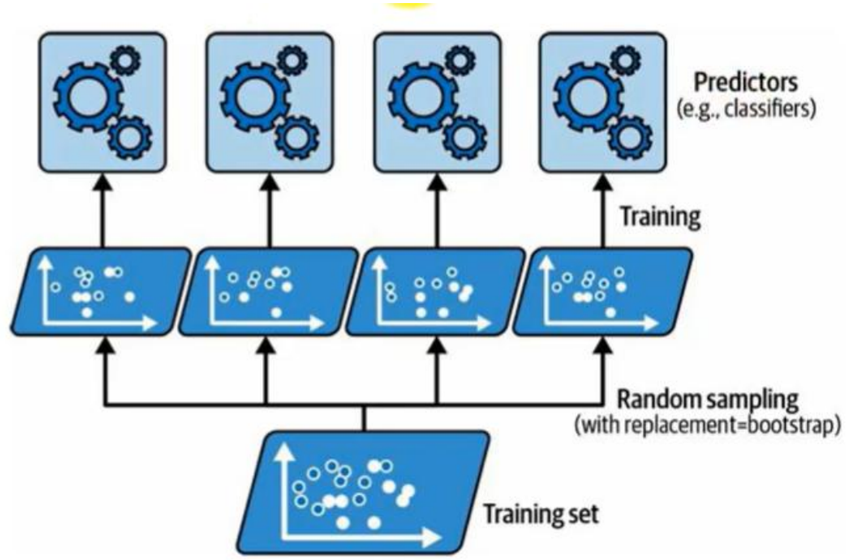
- **Pasting** is similar to bagging but with one key difference:

  sampling without replacement

- This means each data point is used **at most once** in creating the subsets for training the individual models.

Bagging and Pasting

- **Predictors** can all be **trained in parallel**, via different CPU cores or even different servers. Similarly, **predictions** can be made in **parallel**.
- This is one of the reasons bagging and pasting are such popular methods: they **scale very well**.

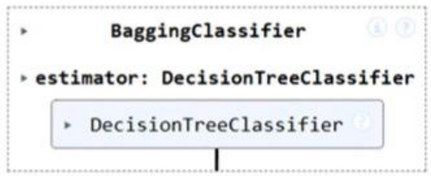


Bagging and Pasting in Scikit-Learn

- Scikit-Learn offers a simple API for both bagging and pasting: **BaggingClassifier** class (or **BaggingRegressor** for regression).

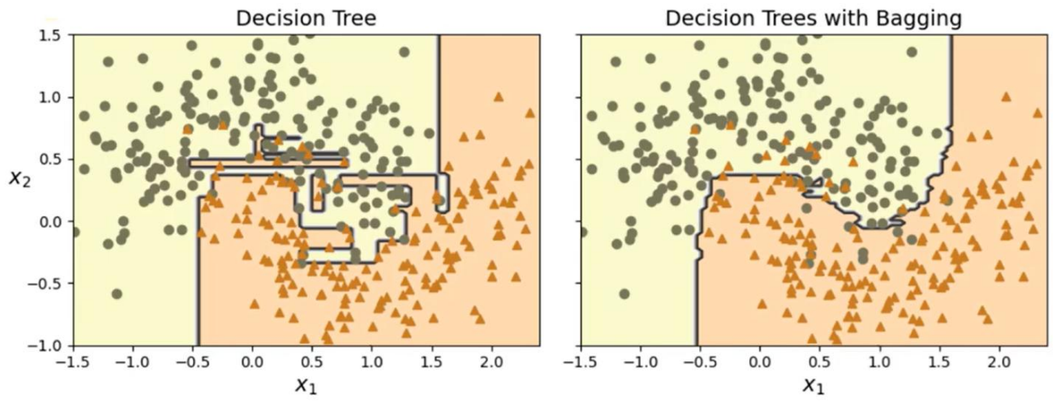
```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
                            max_samples=100, n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)
```



— this is an example of **bagging**, but if you want to use **pasting** instead, just set **bootstrap=False**

Bagging and Pasting



- The ensemble has a comparable bias but a smaller variance (it makes roughly the same number of errors on the training set, but the decision boundary is less irregular).

- Bagging **introduces diversity** in the training process by creating bootstrap samples, which are subsets of the original dataset drawn with replacement.
- This **intentional introduction of diversity** allows each predictor in the ensemble to learn from **slightly different perspectives of the data**.
- As a result, the predictors are less likely to be correlated, which helps in **reducing the ensemble's overall variance**.

- Pasting creates subsets without replacement, typically results in less diversity among predictors.

- While this approach may have a lower bias due to predictors being trained on more representative subsets, the increased correlation among predictors can lead to higher ensemble variance.

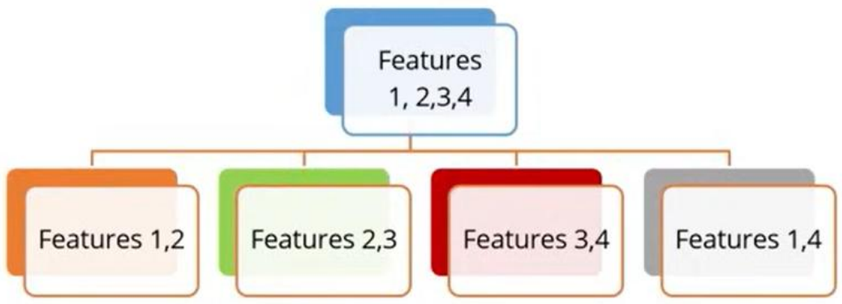
  - This higher variance can reduce the effectiveness of the final model, especially in cases where reducing variance is critical.

- Overall, **bagging** often results in better models, which explains why it's generally preferred.

  - But if you have spare time and CPU power, you can use **cross-validation** to evaluate both bagging and pasting and select the one that works best.

Random Forest

Un bosque aleatorio es una extensión de bagging que denota específicamente el uso de bagging para construir conjuntos de árboles de decisión aleatorios. Esto difiere de los árboles de decisión estándar en que estos últimos muestrean cada característica para identificar la mejor para la división. Por el contrario, los bosques aleatorios muestrean iterativamente subconjuntos aleatorios de características para crear un nodo de decisión.

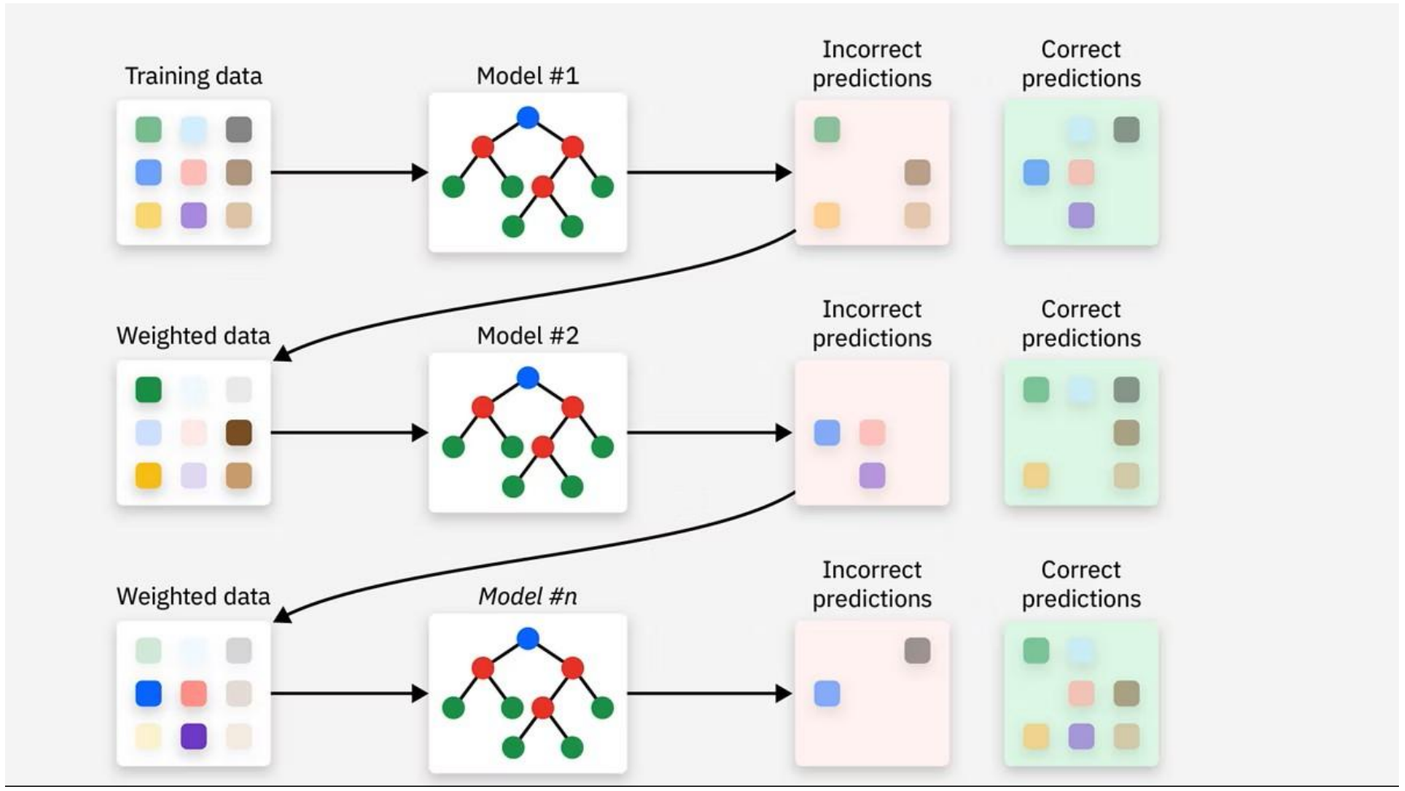


- Random forest technique combines various decision trees to produce a more generalized model.
- Random forests are utilized to produce de-correlated decision trees.
- Random forest creates random subsets of the features.
- Smaller trees are built using these subsets, creating tree diversity.

To overcome overfitting, diverse sets of decision trees are required.

Boosting

Boosting es un método ensemble secuencial en el que múltiples modelos débiles son entrenados de forma iterativa. Cada nuevo modelo busca corregir los errores cometidos por los modelos anteriores. Finalmente, las predicciones de todos los modelos se combinan para obtener un resultado más preciso y robusto.



Modelos dependientes, entrenamiento secuencial, corrección progresiva de errores.

AdaBoost y Gradient Boost

AdaBoost Pondera los errores del modelo. Es decir, al crear una nueva iteración de un conjunto de datos para entrenar al siguiente modelo, AdaBoost añade ponderaciones a las muestras mal clasificadas del modelo anterior, lo que hace que el siguiente modelo priorice esas muestras mal clasificadas.

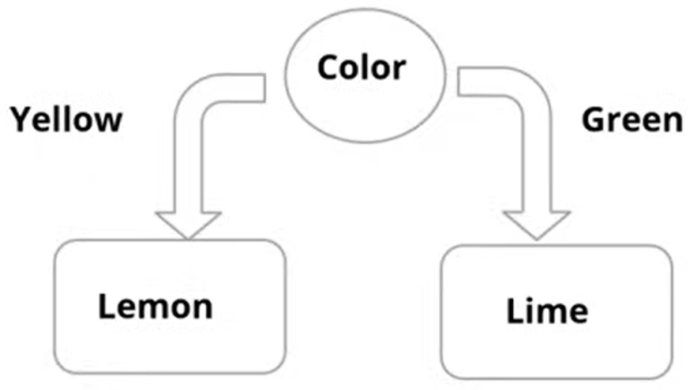
Gradient Boost utiliza los errores residuales al entrenar a los nuevos modelos. En lugar de ponderar las muestras mal clasificadas, el boosting de gradiente utiliza los errores residuales de un modelo anterior para establecer predicciones objetivo para el modelo siguiente. De este modo, intenta cerrar la brecha de error dejada por un modelo.

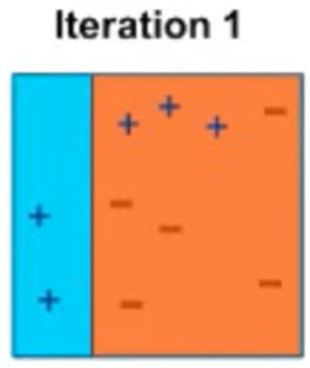
Característica	**AdaBoost**	**Gradient Boosting**
Idea principal	Corrige errores aumentando el peso de muestras mal clasificadas	Corrige errores ajustando residuos mediante gradiente
Tipo de problema inicial	Principalmente clasificación	Clasificación y regresión
Forma de aprendizaje	Repondera observaciones	Optimiza una función de pérdida

Característica	AdaBoost	Gradient Boosting
Entrenamiento secuencial	Sí	Sí
Modelos base comunes	Stumps (árboles muy pequeños)	Árboles de decisión
Sensibilidad al ruido	Alta	Moderada
Riesgo de overfitting	Menor en casos simples	Mayor si no se regula
Complejidad	Más simple	Más complejo
Interpretabilidad	Más fácil	Más difícil
Velocidad	Más rápido	Más lento
Precisión predictiva	Buena	Generalmente superior
Optimización matemática	No basada explícitamente en gradiente	Basada en descenso del gradiente
Variantes modernas	Menos extendido actualmente	Base de XGBoost, LightGBM y CatBoost

AdaBoost Working: Step 1

Assign equal weights to each data point and apply a decision stump to classify them as + (plus) and – (minus).



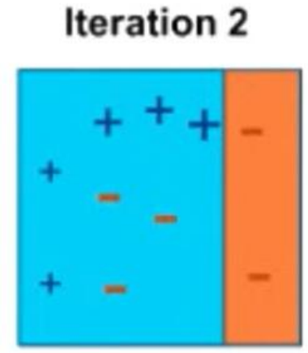


For distinct attributes, the tree consists only of a single interior node.

Now, apply higher weights to incorrectly predicted three + (plus) and add another decision stump.

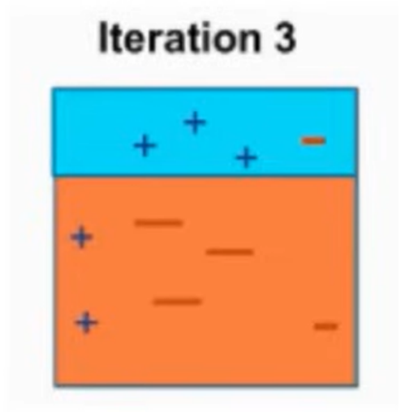
AdaBoost Working: Step 2

- The size of three incorrectly predicted + (plus) is mad bigger than the rest of the data points.
- The second decision stump (D2) will try to predict them correctly.
- Now, vertical plane (D2) has classified three mis-classified + (plus) correctly.
- D2 has also caused mis-classification errors to three – (minus).



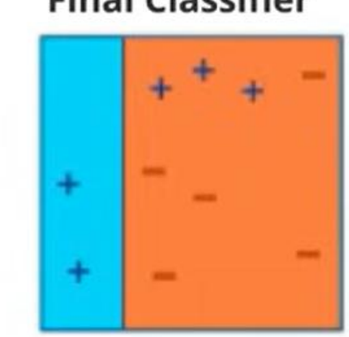
AdaBoost Working: Step 3

- D3 adds higher weights to three – (minus).
- A horizontal line is generated to classify + (plus) and – (minus) based on higher weight of mis-classified observation.



AdaBoost Working: Step 4

- D1, D2, and D3 are combined to form a strong prediction that has a more complex rule than individual weak learners.



AdaBoost Algorithm

- A weak classifier is prepared on the training data using the weighted samples.
- Only binary classification problems are supported.
- Every decision stump makes one decision on one input variable and outputs a +1.0 or -1.0 value for the first- or second-class value.

Misclassification rate

error = (correct – N)/N

AdaBoost Algorithm

STEP 1

Initially each data point is weighted equally with weight

$Wi = 1/n$

where n is the number of samples

STEP 2

A classifier 'H1' is picked up that best classifies the data with minimal error rate

STEP 3

The weighing factor α is dependent on errors (ϵ_t) caused by the H1 classifier

$$\alpha^t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$$

STEP 4

Weight after time t is given as :

$$\frac{w_i^{t+1}}{Z} e^{-\alpha t . h1(x) . y(x)}$$

where z is the normalizing factor, h1(x).y(x) is sign of the current output

AdaBoost Flowchart

AdaBoost selects a training subset randomly.

↓

It iteratively trains the AdaBoost machine learning model.

↓

It assigns a higher weight to wrongly classified observations.

↓

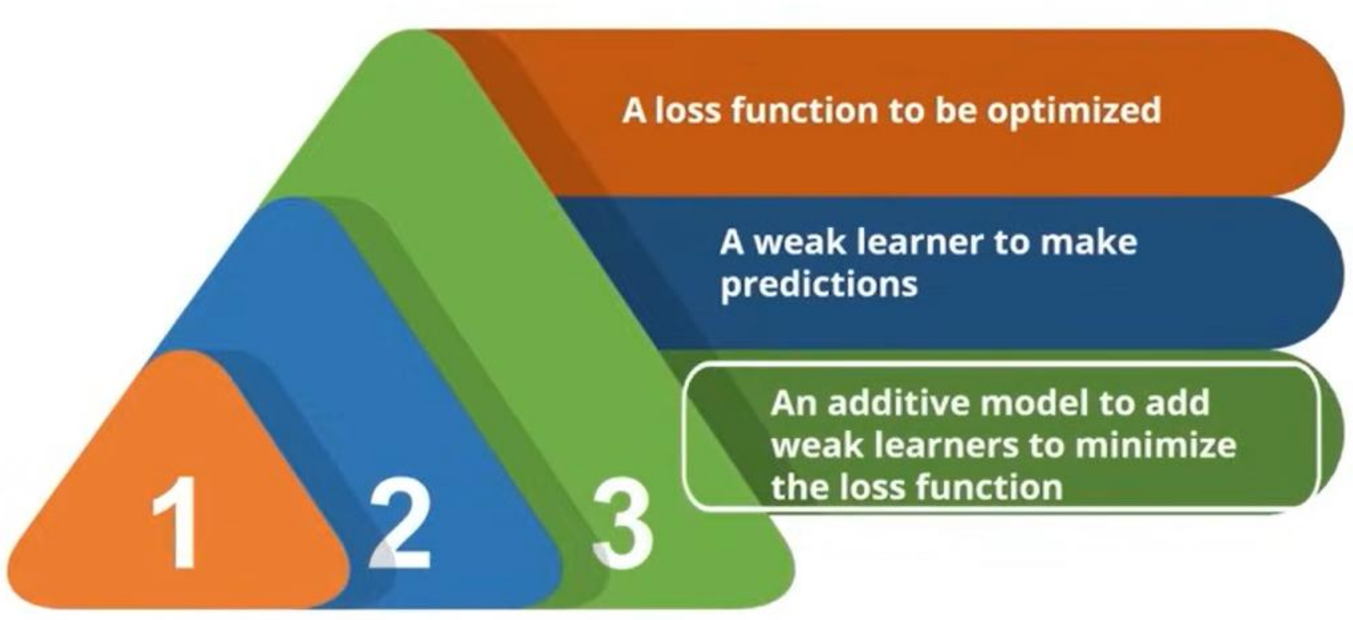
It assigns weight to the trained classifier in each iteration according to the accuracy of the classifier.

↓

This process iterates until the complete training data fits without any error.

Gradient Boosting (GBM)

- Gradient boosting trains several models in a very gradual, additive, and sequential manner.
- GBM minimizes the loss function (MSE) of a model by adding weak learners using a gradient descent procedure.
- Gradient boosting involves three elements:

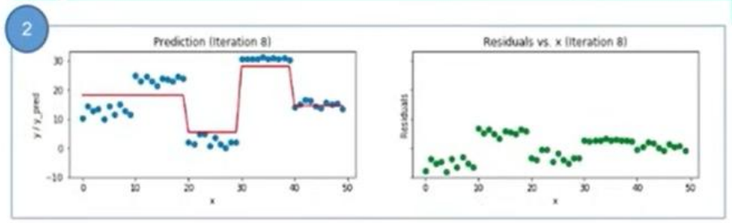


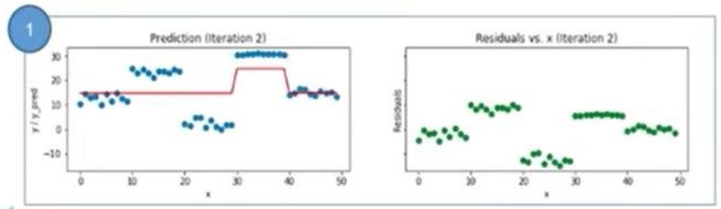
GBM Mechanism

- Gradient boosting trains many models sequentially.
- Each new model gradually minimizes the loss function of the whole system using the Gradient Descent method.
- In y = ax + b + e, e needs special attention because it is an error term.
- The learning procedure consecutively fits new models.
- The principle idea behind this algorithm is to construct new base learners.

GBM Mechanism

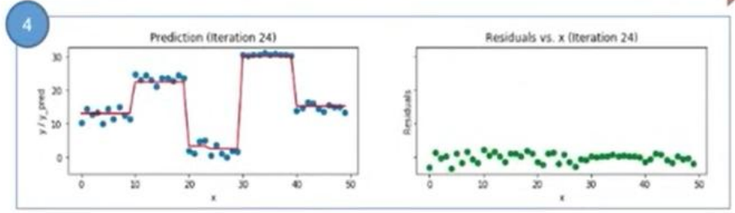
01 GBM predicts the residuals or errors of prior models and then sums them to make the final prediction.

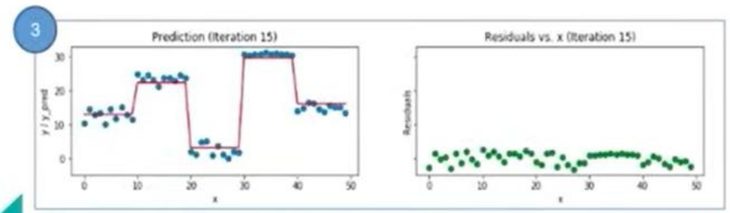




02 One weak learner is added at a time and existing weak learners in the model are left unchanged.

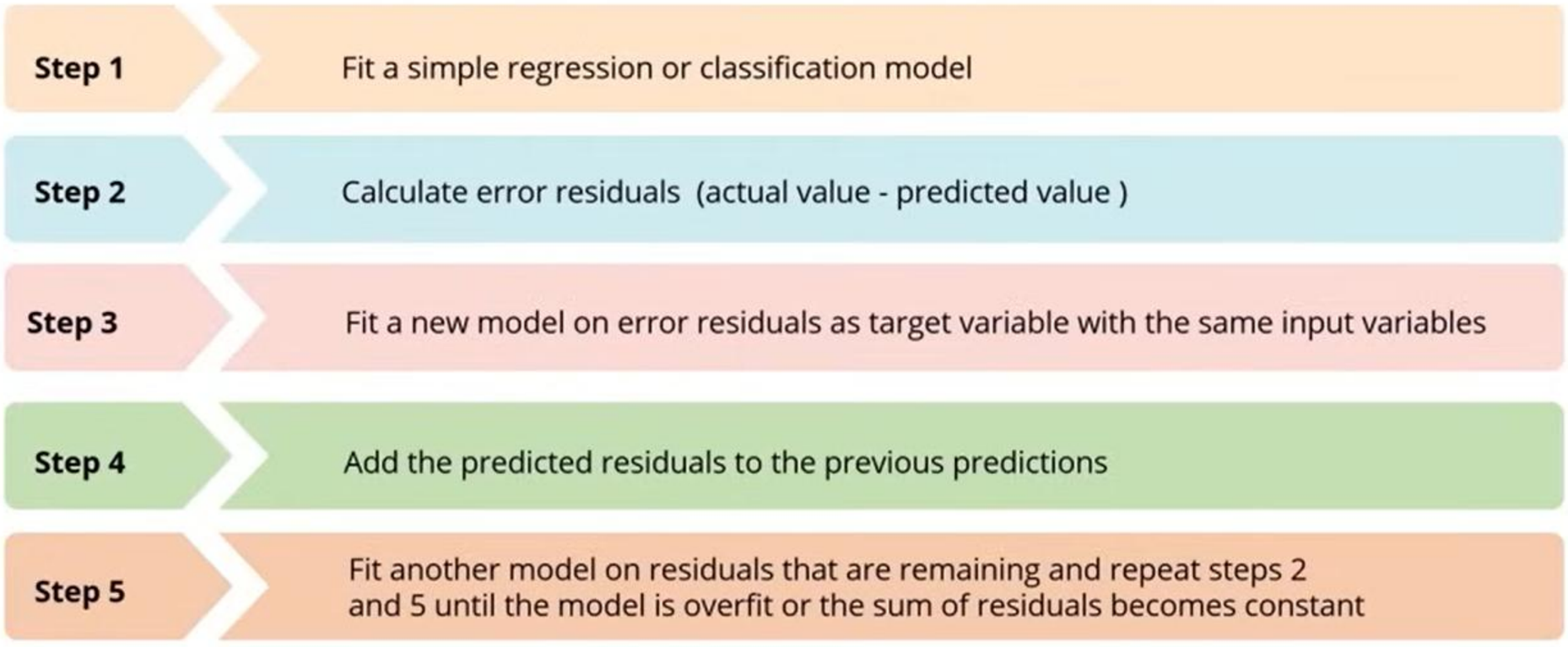
03 GBM repetitively leverages the patterns in residuals and strengthens a model with weak predictions.





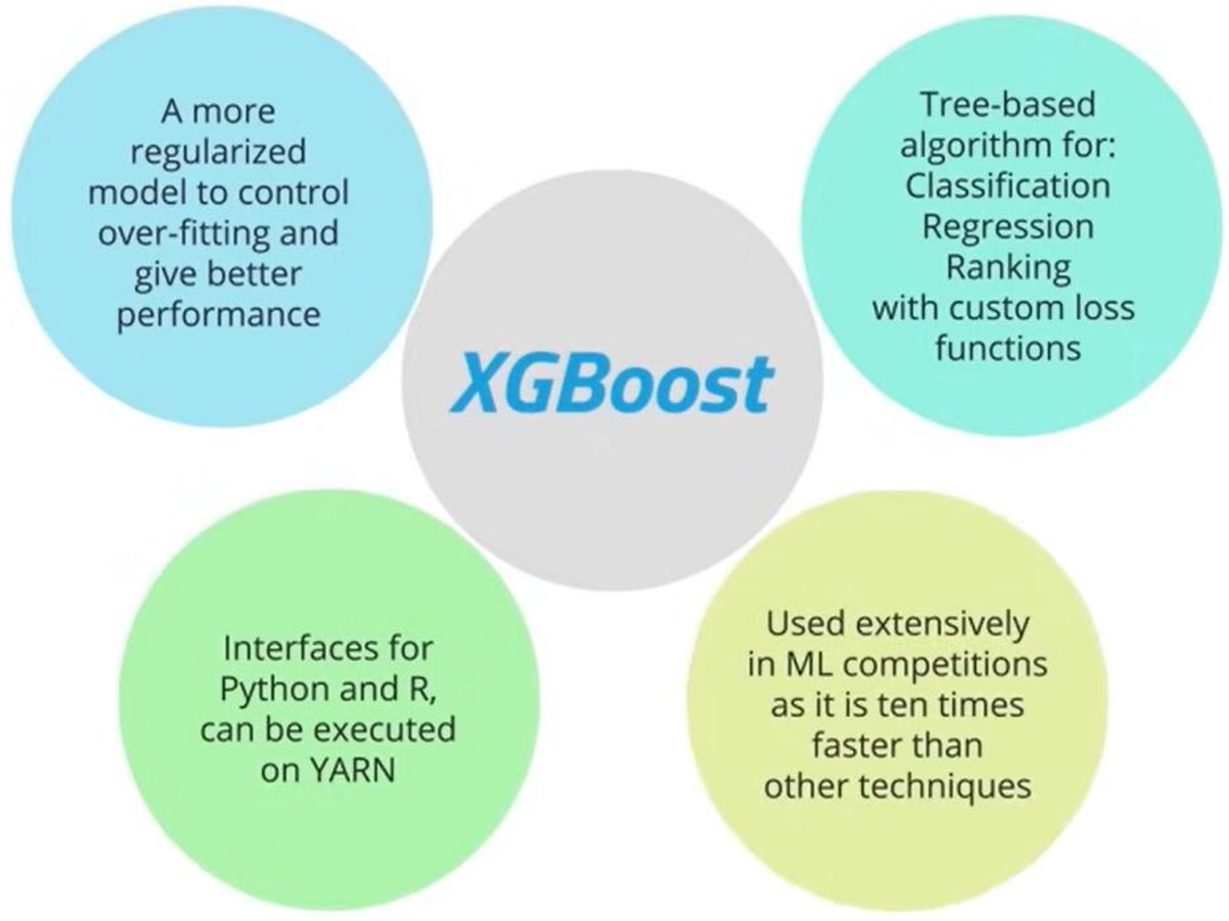
04 Modeling is stopped when residuals do not have any pattern that can be modeled.

GBM Algorithm



XGBoost

eXtreme Gradient Boosting is a library for developing fast and high-performance gradient boosting tree models.



XGBoost Library Features

XGBoost library features tools are built for the sole purpose of model performance and computational speed.

01

SYSTEM

Parallelization
Tree construction using all CPU cores while training

Distributed Computing
Training very large models using a cluster of machines

Cache Optimization
Data structures make best use of hardware

02

ALGORITHM

Sparse Aware
Automatic handling of missing data values

Block Structure
Supports the parallelization of tree construction

Continued Training
To boost an already fitted model on new data

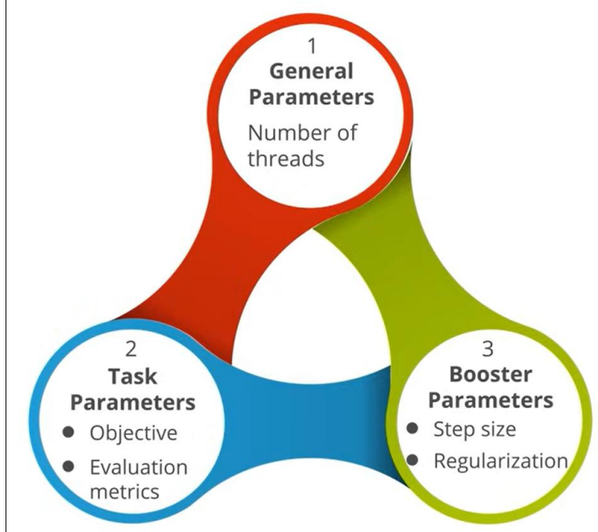
03

MODELS

Gradient Boosting
Gradient boosting machine algorithm including learning rate

Stochastic Gradient Boosting
Sub-sampling at the row, column and column per split levels

Regularized Gradient Boosting
With both L1 and L2 regularization

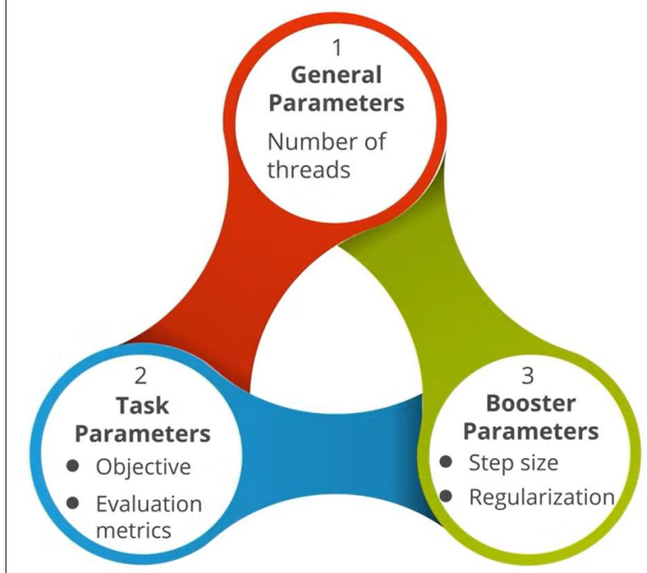


General Parameters

General parameters guide the overall functioning of XGBoost.

- nthread
  - Number of parallel threads
  - If no value is entered, algorithm automatically detects the number of cores and runs on all the cores
- booster
  - gbtree: tree-based model
  - gblinear: linear function
- Silent [default = 0]
  - If set to 1, no running messages will be printed.

Hence, keep it '0' as the messages might help in understanding the model

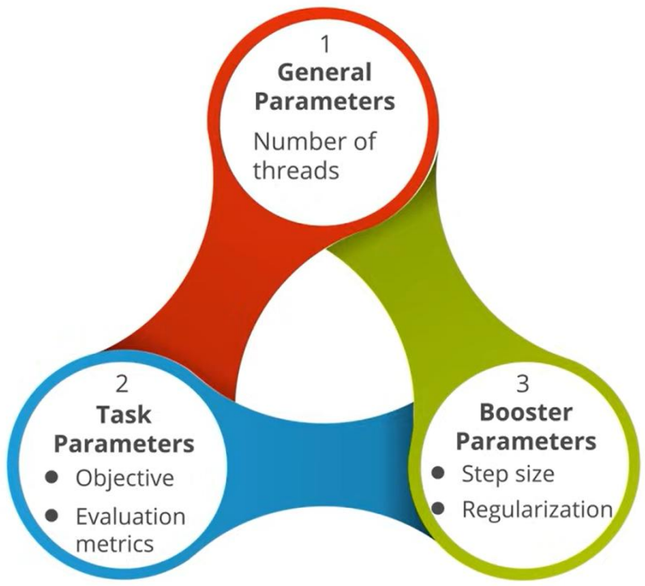


Boster parameters (part 1/3)

Booster parameters guide individual booster (Tree/Regression) at each step.

Parameters for tree booster

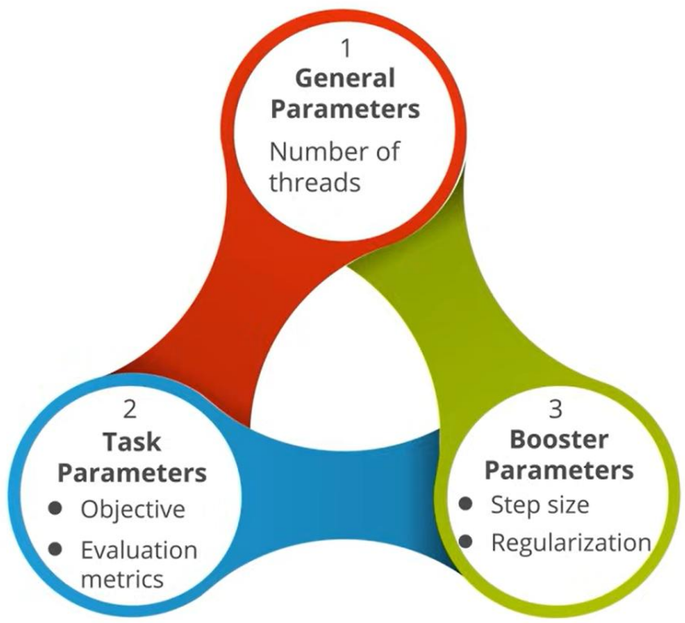
- eta
  - Step size shrinkage is used in update to prevent overfitting
  - Range in [0,1], default = 0.3
- gamma
  - Minimum loss reduction required to make a split
  - Range [0,∞], default = 0
- max_depth
  - Maximum depth of a tree
  - Range [1, ∞], default = 6
- min_child_weight
  - Minimum sum of instance weight needed in a child
  - If the tree partition results in a leaf node with sum of instance weight less that min_child_weight, then the building process will give up further partitioning
  - Range [0, ∞], default 1



Boster parameters (part 2/3)

Parameters for tree booster

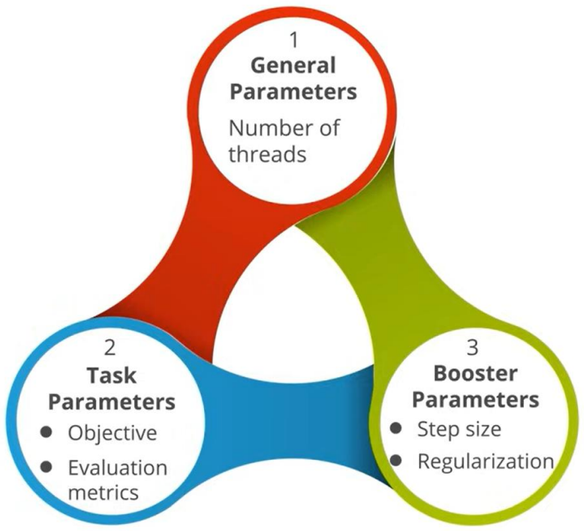
- max_delta_step
  - Maximum delta step allowed in each tree's weight estimation
  - Range in [0, ∞], default = 0
- subsample
  - Subsample ratio of the training instance
  - Range [0, 1], default = 1
- Colsample_bytree
  - Subsample ratio of columns when constructing each tree
  - Range [0, 1], default = 1



Boster parameters (part 3/3)

Parameters for linear booster

- lambda
  - L2 regularization term on weights
  - default = 0
- alpha
  - L1 regularization term on weights
  - default = 0
- Lambda_bias
  - L2 regularization term on bias
  - default = 0



Task parameters

Task parameters guide the optimization objective to be calculated at each step.

1. Objectives [default = reg:linear]

- "binary:logistic": logistic regression for binary classification, output is probability not class
- "multi:softmax": multiclass classification using the softmax objective, need to specify num_class

2. Evaluation Metrics: A default metric will be assigned according to the objective (rmse for regression, error for classification, and mean avg precision for ranking)

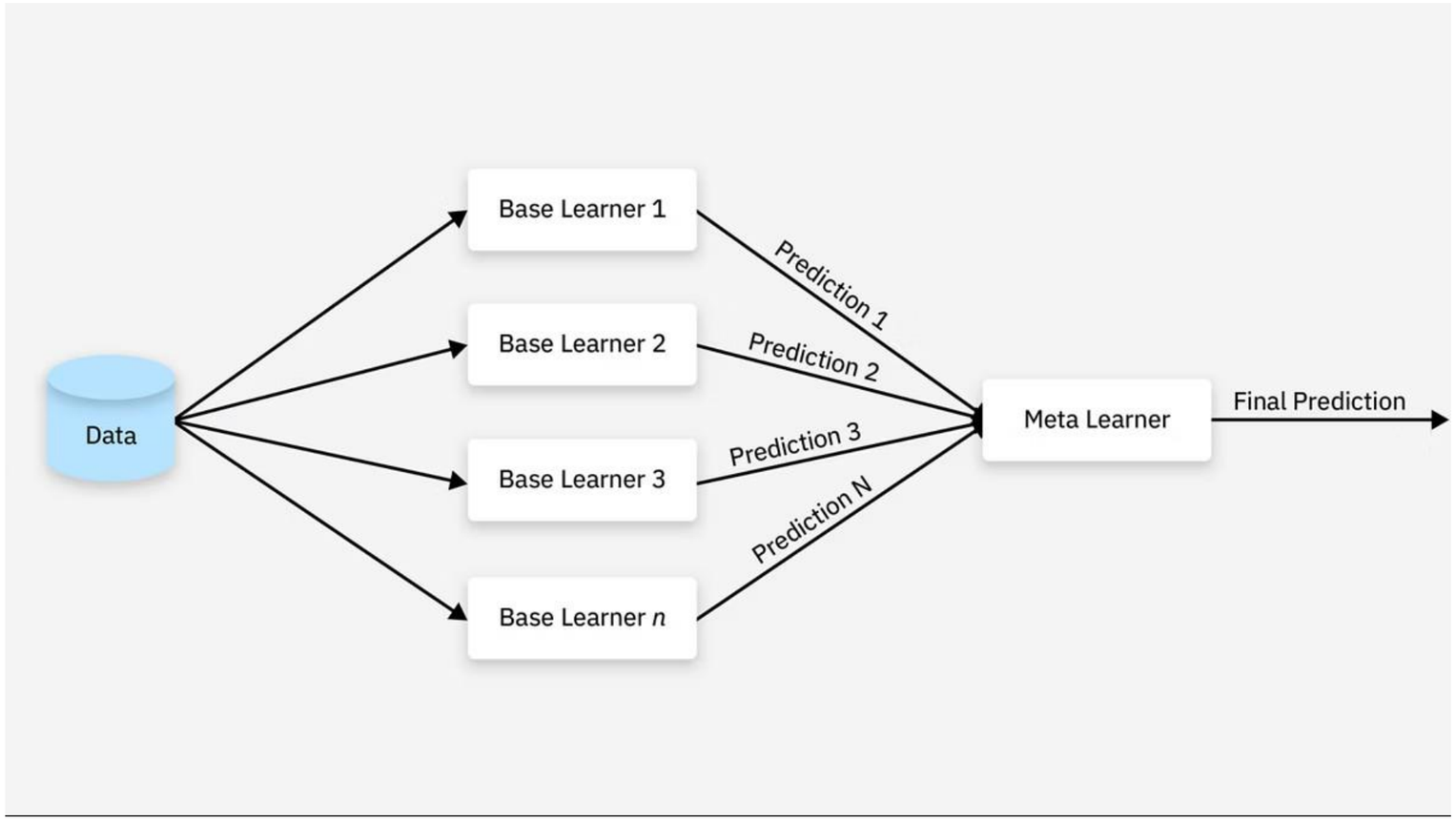
- "rmse"
- "logloss"
- "error"
- "auc"

Característica	AdaBoost	Gradient Boosting	XGBoost
Idea principal	Reforzar muestras mal clasificadas	Minimizar errores residuales	Optimización avanzada de Gradient Boosting
Tipo de aprendizaje	Secuencial	Secuencial	Secuencial
Cómo corrige errores	Ajusta pesos de observaciones	Aprende residuos	Aprende residuos optimizados
Base matemática	Reponderación adaptativa	Descenso del gradiente	Descenso del gradiente regularizado
Modelos base típicos	Árboles débiles (stumps)	Árboles de decisión	Árboles optimizados
Precisión	Buena	Alta	Muy alta
Velocidad	Rápida	Moderada	Muy rápida y optimizada
Riesgo de overfitting	Moderado	Alto si no se regula	Menor gracias a regularización
Regularización	No	Limitada	Sí (L1 y L2)
Manejo de valores faltantes	Limitado	Moderado	Muy bueno
Paralelización	Baja	Baja	Alta
Escalabilidad	Media	Media	Alta
Sensibilidad al ruido	Alta	Moderada	Menor
Complejidad	Baja	Media	Alta
Uso actual	Académico/didáctico	Muy utilizado	Dominante en competiciones y aplicaciones reales

Stacking o blending de modelos

El apilamiento (stacking), o generalización apilada es un método paralelo heterogéneo que ejemplifica lo que se conoce como metaaprendizaje. El metaaprendizaje consiste en entrenar a un metaaprendiz a partir de los resultados de varios aprendices base. El stacking entrena específicamente a varios modelos base a partir del mismo conjunto de datos utilizando un algoritmo de entrenamiento diferente para cada modelo.

Cada modelo base hace predicciones en un conjunto de datos no visto. Estas primeras predicciones del modelo se compilan y se utilizan para entrenar un modelo final, que es el metamodelo.



Stacking o blending de modelos

En stacking se debe tener en cuenta la importancia de utilizar un conjunto de datos distinto del utilizado para entrenar a los modelos aprendices de base, con el fin de entrenar al metaaprendiz.

Utilizar el mismo conjunto de datos para entrenar a los aprendices base y al metaaprendiz puede dar lugar a un sobreajuste. Esto puede requerir excluir instancias de datos de los datos de entrenamiento del aprendiz base para que sirvan como datos de prueba, que a su vez se convierten en datos de entrenamiento para el metaaprendiz.

La bibliografía suele recomendar técnicas como la validación cruzada para garantizar que estos conjuntos de datos no se solapen.

Comparación Bagging, Boosting y Stacking

Característica	Bagging (ej. Random Forest)	Boosting (ej. XGBoost)	Stacking
Tipo de modelos	Homogéneos (mismo algoritmo)	Homogéneos (mismo algoritmo)	Heterogéneos (diferentes algoritmos)
Entrenamiento	En paralelo sobre muestras aleatorias	En secuencia (corrigiendo errores previos)	En paralelo (nivel 0) y secuencial (nivel 1)
Método de unión	Promedio o votación democrática	Combinación ponderada predefinida	Un meta-modelo aprende la mejor combinación

Práctica de experimentación 1

Predicción de pacientes mujeres con diabetes, con base en ciertas medidas de diagnóstico que constan en el dataset (ejm. Embarazos, BMI, nivel de insulina, edad, entre otras), por medio de clasificación AdaBoost y XGBoost.

Entorno de desarrollo: Jupyter Notebook

Código:

```
import pandas
from sklearn import model_selection
from sklearn.ensemble import AdaBoostClassifier
```

```
dataframe = pandas.read_csv('pima-indians-diabetes.csv')
array = dataframe.values
X = array[:,0:8]
Y = array[:,8]
seed = 7
num_trees = 30
```

```
kfold = model_selection.KFold(n_splits=10,random_state=seed)
model = AdaBoostClassifier(n_estimators=num_trees,random_state=seed)
results = model_selection.cross_val_score(model,X,Y,cv=kfold)
print(results.mean)
```

```
from sklearn import svm
from xgboost import XGBClassifier
clf = XGBClassifier()

seed =7
num_trees=30
kfold = model_selection.KFold(n_splits=10,random_state=seed)
model = XGBClassifier(n_estimators=num_trees,random_state=seed)
results = model_selection.cross_val_score(model,X,Y,cv=kfold)
print(results.mean())
```

```
from sklearn.ensemble import RandomForestClassifier
rf_class = RandomForestClassifier(n_estimators=10)
```

```
from sklearn.model_selection import cross_val_score
print(cross_val_score(rf_class,data_input,data_output,scoring='accuracy',cv=10))
```

```
accuracy = cross_val_score(rf_class,data_input,data_output,scoring='accuracy',cv=10).mean()*100
print('Accuracy of Random Forests is:',accuracy)
```

Analizar los resultados obtenidos.

Práctica de experimentación 2

Seleccionar el mejor modelo RANDOM FOREST, considerando la técnica de división del data set cross-validation, en un problema de clasificación de la base de datos IRIS.

```python
from sklearn.datasets import load_iris
```

```python
iris_data = load_iris()
print(iris_data)
```

```python
data_input = iris_data.data
data_output = iris_data.target
```

```python
print(data_input)
```

```python
print(data_output)
```

```python
from sklearn.model_selection import KFold
kf = KFold(n_splits=5,shuffle=True)
```

```python
print("Train Set         Test Set      ")
for train_set,test_set in kf.split(data_input):
    print(train_set,test_set)
```

```python
from sklearn.ensemble import RandomForestClassifier
rf_class = RandomForestClassifier(n_estimators=10)
```

```python
from sklearn.model_selection import cross_val_score
print(cross_val_score(rf_class,data_input,data_output,scoring='accuracy',cv=10))
```

```python
accuracy = cross_val_score(rf_class,data_input,data_output,scoring='accuracy',cv=10).mean()*100
print('Accuracy of Random Forests is:',accuracy)
```

Analizar los resultados obtenidos.